Coronel, Joshua S.
C204

Finals Task 3. Simple Polymorphism

Problem. Chirp and Tweet
Create a simple program to demonstrate basic polymorphism with bird sounds

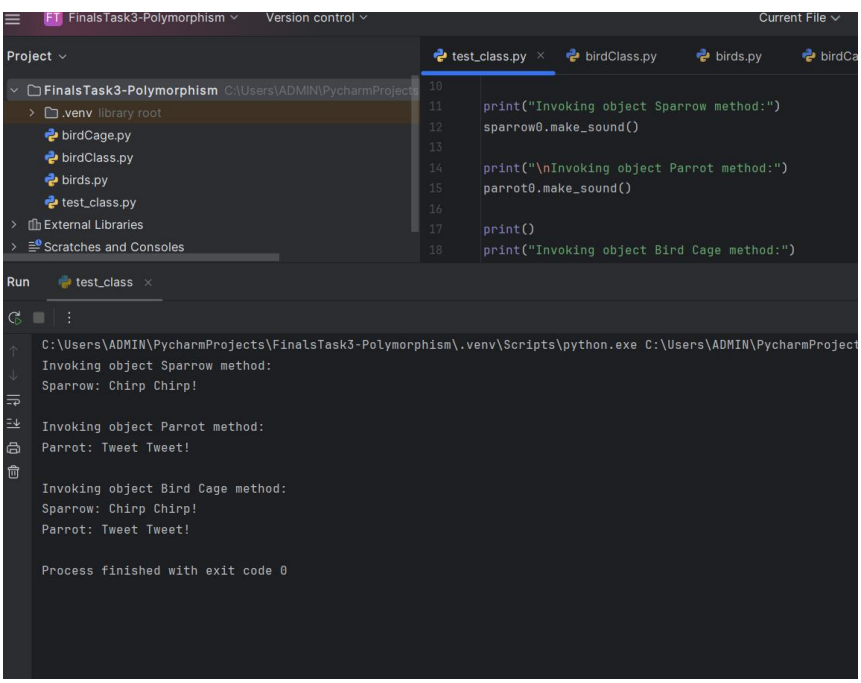
Class Bird:
method make_sound() – Abstract method that represents making sound

Class Sparrow(Extends Bird):
method: def make_sound(self) – overrides the make_sound method from base class Bird
prints sound: “Chirp Chirp”

Class Parrot(Extends Bird):
method: def make_sound(self) – overrides the make_sound method from base class Bird
prints sound: “Tweet tweet!”

Class Bird Cage:
Methods: def make_sound – accept a list of birds object and call the make sound method on each birds

Output:



Code:

testClass:

```python
from birdClass import Bird
from birds import Sparrow,Parrot
from birdCage import Birdcage

sparrow0 = Sparrow()
sparrow0.name = "Sparrow"
parrot0 = Parrot()
parrot0.name = "Parrot"

print("Invoking object Sparrow method:")
sparrow0.make_sound()

print("\nInvoking object Parrot method:")
parrot0.make_sound()

print()
print("Invoking object Bird Cage method:")
cage0 = Birdcage([sparrow0,parrot0])
cage0.make_sound()
```

birdClass:

```python
class Bird(ABC):  5 usages
    def __init__(self, name=None):
        self.__name = name

    @property  5 usages
    def name(self):
        return self.__name

    @name.setter  4 usages
    def name(self, name):
        self.__name = name

    @abstractmethod  1 usage (1 dynamic)
    def make_sound(self):
        pass
```

birds: contains class Sparrow and Class Parrot

```python
from birdClass import Bird

class Sparrow(Bird):  3 usages
    def __init__(self, name=None):
        super().__init__(name)

    def make_sound(self):  2 usages (1 dynamic)
        print(f"{self.name}: Chirp Chirp!")


class Parrot(Bird):  3 usages
    def __init__(self, name = None):
        super().__init__(name)

    def make_sound(self):  2 usages (1 dynamic)
        print(f"{self.name}: Tweet Tweet!")
```

```python
from birdClass import Bird
from birds import Sparrow
from birds import Parrot

class Birdcage():  2 usages
    def __init__(self, birds:list ):
        self.birds = birds


    def make_sound(self):  2 usages (1 dynamic)
        for bird in self.birds:
            bird.make_sound()
```

birdCage class